

计算机组成原理实验报告

实验时间	2024.6.10	实验名称	微程序控制器设计
学号	2220222043	姓名	海子亮
专业	软件工程	实验成绩	

一、 实验目的

- 1. 掌握微程序控制器的组成原理和工作过程；
- 2. 理解微指令和微程序的概念，理解微指令与指令的区别与联系；
- 3. 理解“微程序”设计思想，掌握微程序控制器的结构和设计方法。

二、 实验环境

Proteus 8 Professional

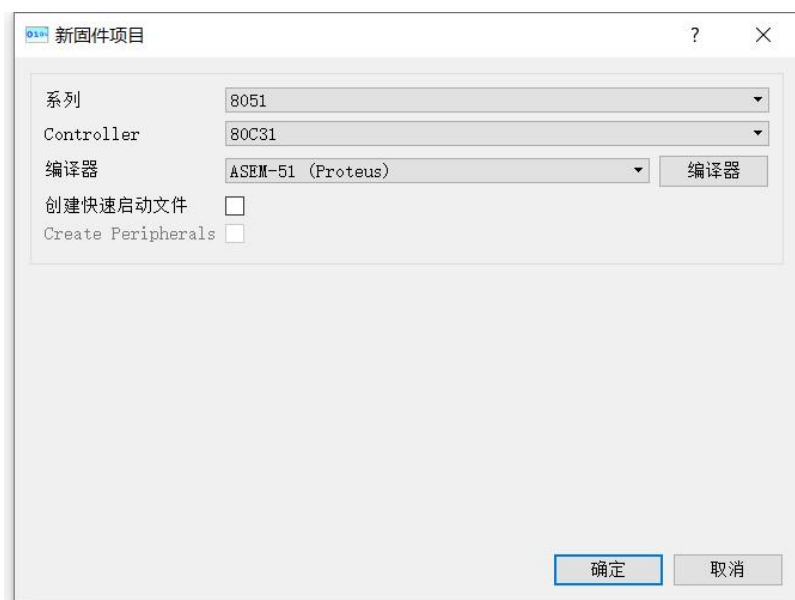
三、 实验内容

- 1、根据微指令代码表编写微程序，编译并生成三个 HEX 文件，分别烧写到控制存储器 EPROM1、EPROM2 及 EPROM3。
- 2、完成 ROM 部分的电路原理图，并在实验报告中记录向其中烧录微程序的操作过程。
- 3、基于“微程序”原理，设计一个“最小版本”CPU, 实现跳转功能写成.asm，转为.hex 烧写到 PROGRAM 芯片中。

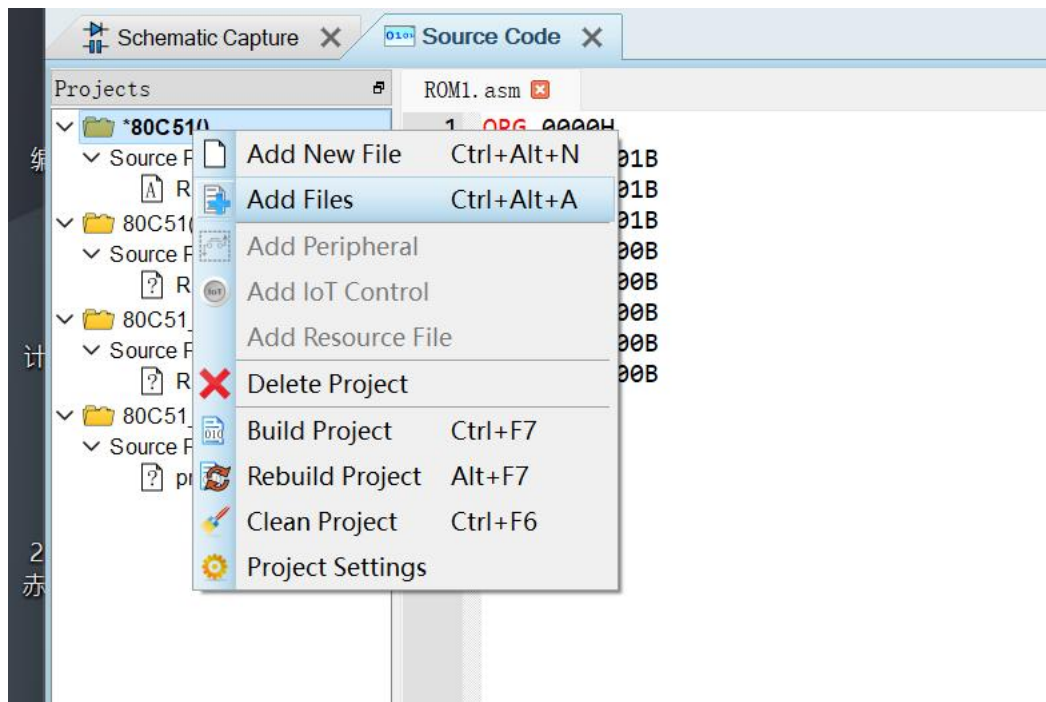
四、 仿真过程

注：上述代码中的语句“ORG xxxxH”规定该语句后所跟数组存储的首地址，数组末尾必须以其他 ORG 语句或“END”作为结束。在 asm 文件中可以使用多个“ORG”语句来规定在存储器的不同位置存放不同长度的数组。在“ORG”语句后的“DB xxxxxxxxB”或“DB xxH”语句都表示一个存储单元存放的 8 位数据，前者是二进制，x 表示 0 或 1；后者是十六进制，x 表示 09 和 AF（注：若十六进制数据 xxH 大于 A0H，则要写成 0xxH，例如“DB 0A0H”）。

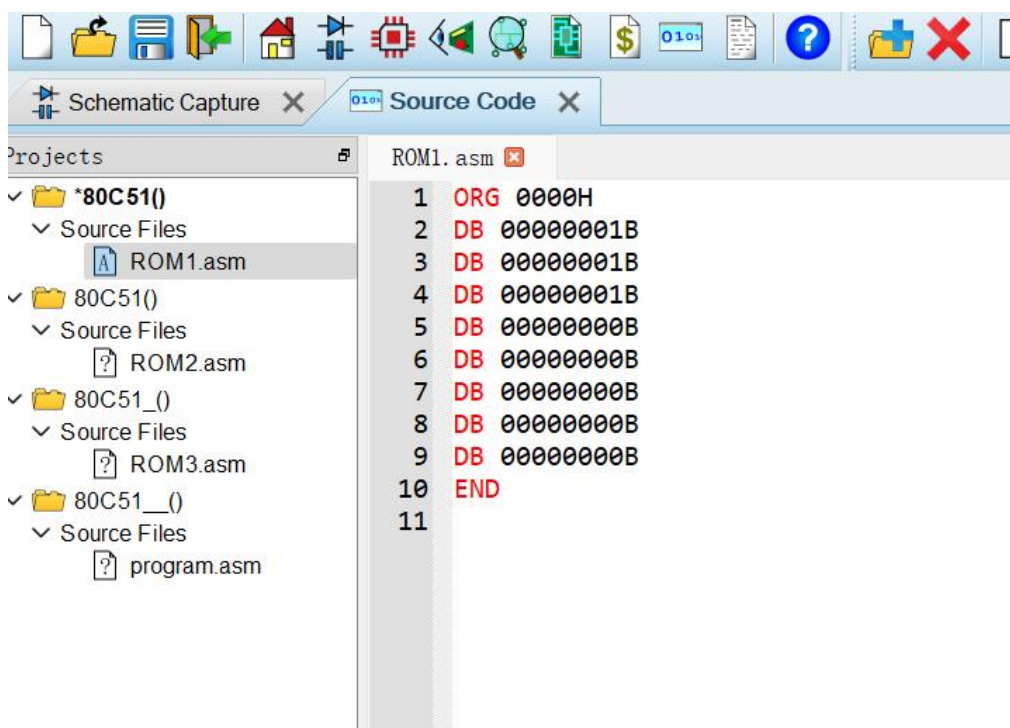
（2）在 proteus 仿真环境的菜单栏里点击 Source Code 图标，弹出 Source Code 标签页，如果该项目没有打开过 asm 文件，则打开的标签页为空，需要在菜单栏里选择 Source Code->Project->新建工程，弹出的对话框如下图所示。此处，在系列选项中选择 8051，Controller 选项中选择 80C51，则编译器选项会自动选择 ASEM-51；注意：下方创建快速启动文件选项不勾选！最后按下“确定”按钮。



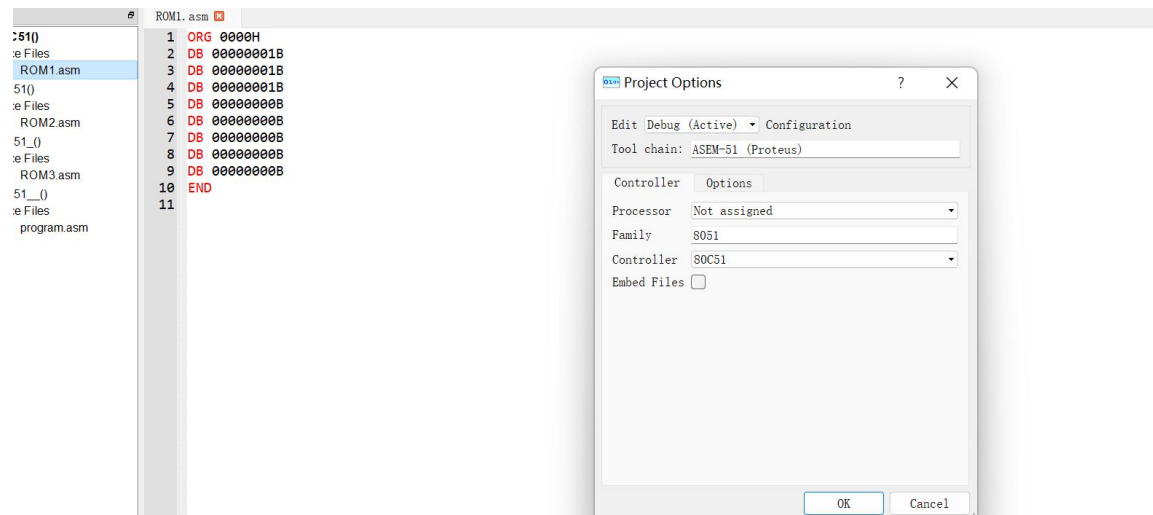
（3）如下图所示，标签页上会出现“” *80C51()”的新 project 后，右键单击下方的 Source Files，在弹出菜单中选择 Add files（或 Import Existing File）选项，在弹出的视窗中选择所需的 asm 文件，按确定就添加到当前的 project 中。



(4) 在 project 中已经添加的 asm 文件上双击，右侧会打开 asm 文本内容，如下图所示。如果该 project 之前已经添加且编译过 asm 文件，则弹出的 Source Code 标签页右侧会直接显示 asm 文本内容，可以直接对 asm 文本内容进行修改和保存。如果要更换 asm 文件，可以右键单击 asm 文件，菜单列表中选择 Remove file 删除当前 asm 文件，然后选择 Add files（或 Import Existing File）选项，在弹出的视窗中重新选择新的 asm 文件。



(5) 如下图所示，若第一次编译 asm 文件，则右键单击 asm 文件，在弹出菜单点击“工程设置”，弹出框的“控制器”选项选择 80C51，注：要去掉勾选“嵌入式文件”！确保编译生成的 hex 文件在 project 当前目录下，最后点击确定选项。



(6) 然后，右键单击 asm 文件，在弹出框内点击 Build Project 执行编译。编译成功后，在下方的显示栏里会出现“编译成功”字样。此时，在项目当前文件夹的子文件夹 80C51 的孙文件夹 Debug 里面，就有编译后生成的 hex 二进制文件（请注意看文件的修改日期，确认是最近编译的文件）。该 hex 文件默认编译后的文件名为 Debug.hex。

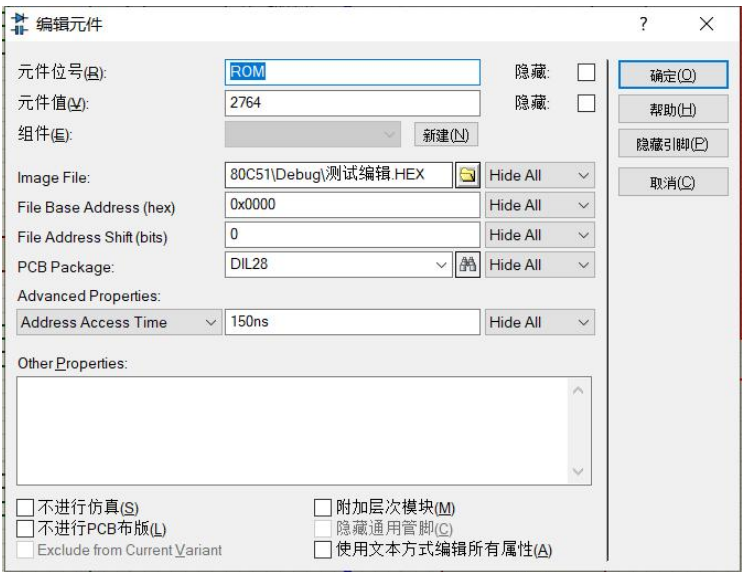
注意：一个良好的编程习惯是每次编译生成 hex 文件后，就重命名为跟 asm 源程序一致的文件名，然后与同名 asm 源程序放在另外的固定目录下。否则下一次编译其他 asm 源程序的时候，会在相同路径生成重名的 Debug.hex，把以前编译的 hex 文件覆盖。



(7) 双击 ROM 芯片，弹出如下图所示的对话框：在 Image File 中选择所需烧写的 hex 文件，点击 OK，ROM 加载成功，操作结束。

注意：尽量避免直接加载路径“\80C51\Debug”里的 Debug.hex 文件，因为每一次编译 asm 源程序，Debug.hex 都会自动刷新。但是，相同的文件名很容易混淆，

难以确定加载的 hex 文件与哪个 asm 源文件对应。最好在每次编译完成后，把 Debug.hex 文件从路径 “\80C51\Debug” 里取出，另存在其他文件夹中，并且根据源文件 asm 的内容重命名编译生成的 hex 文件。



烧录结果如下：



Memory Contents - ROM

[illegible]

rom.asm

文件 编辑 查看

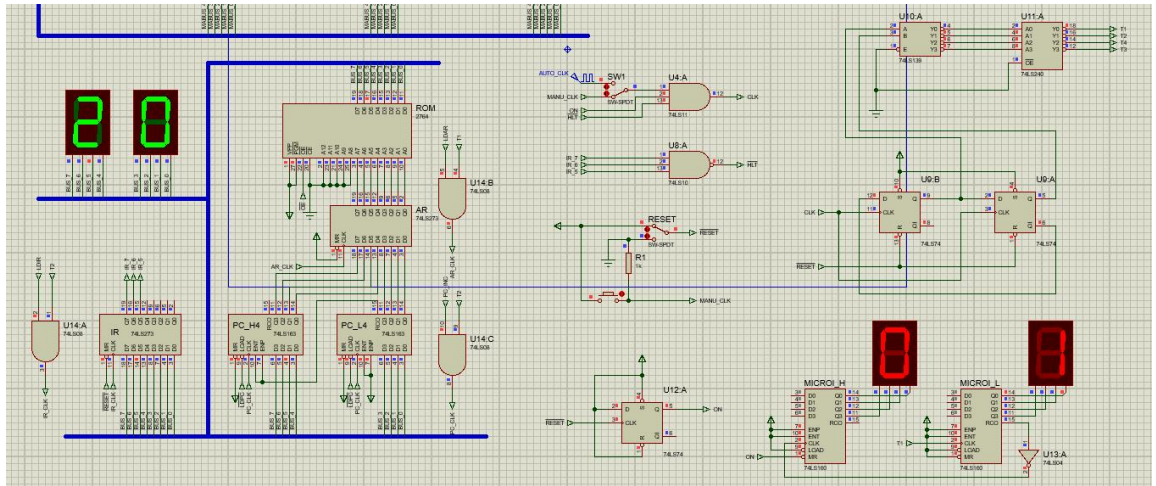
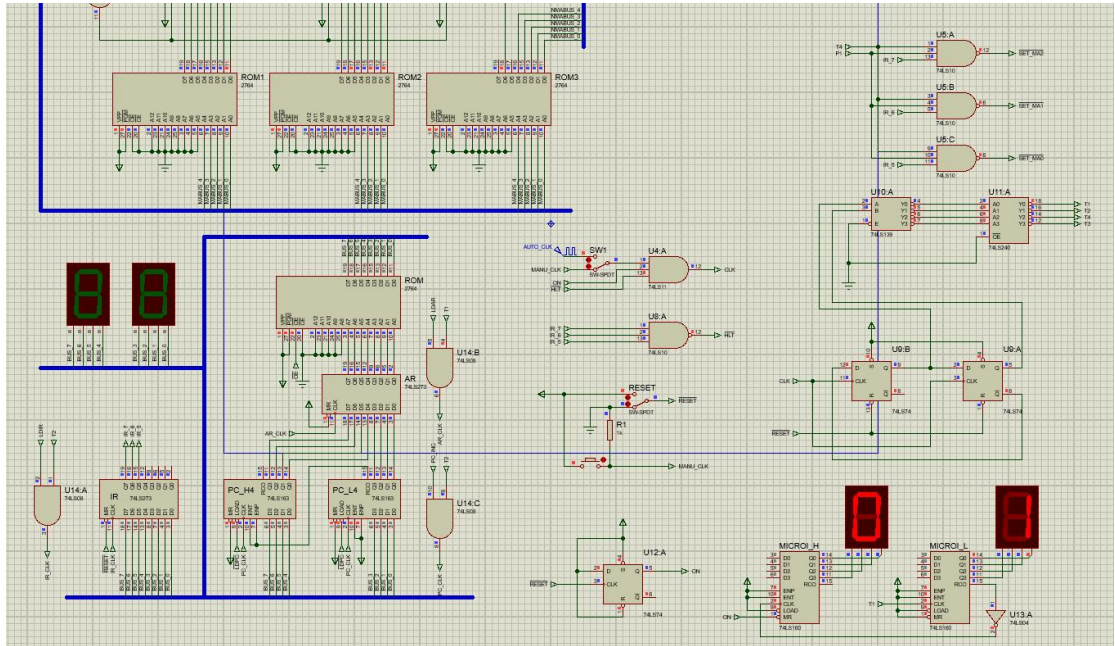
```

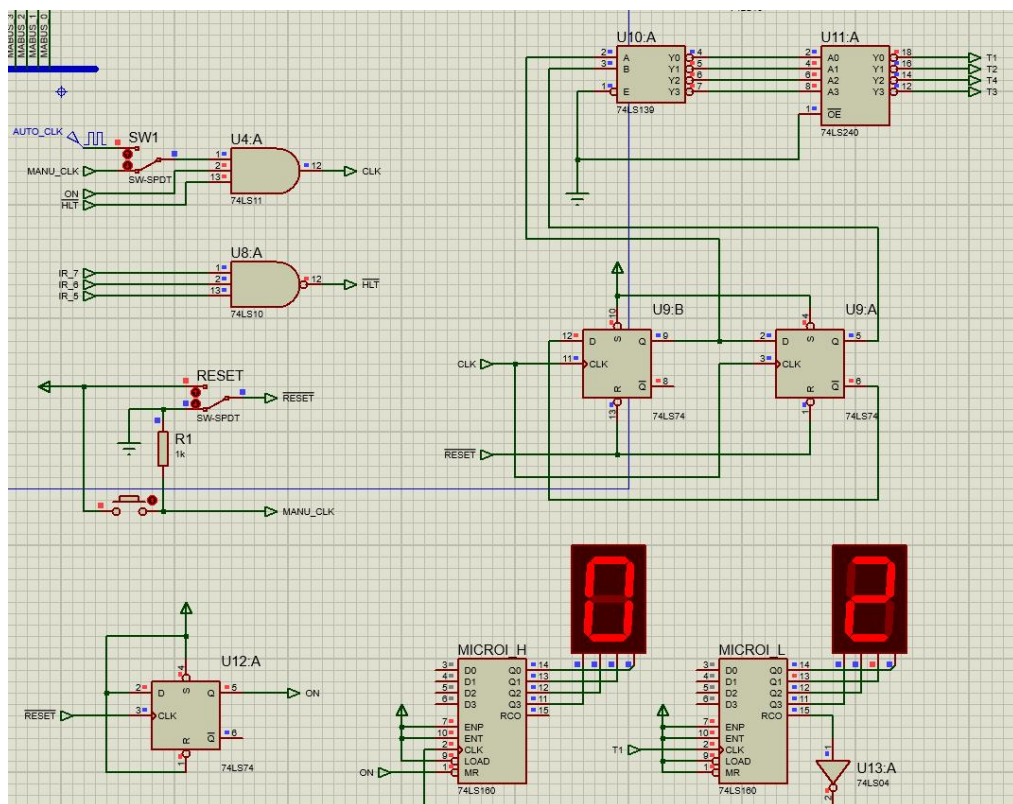
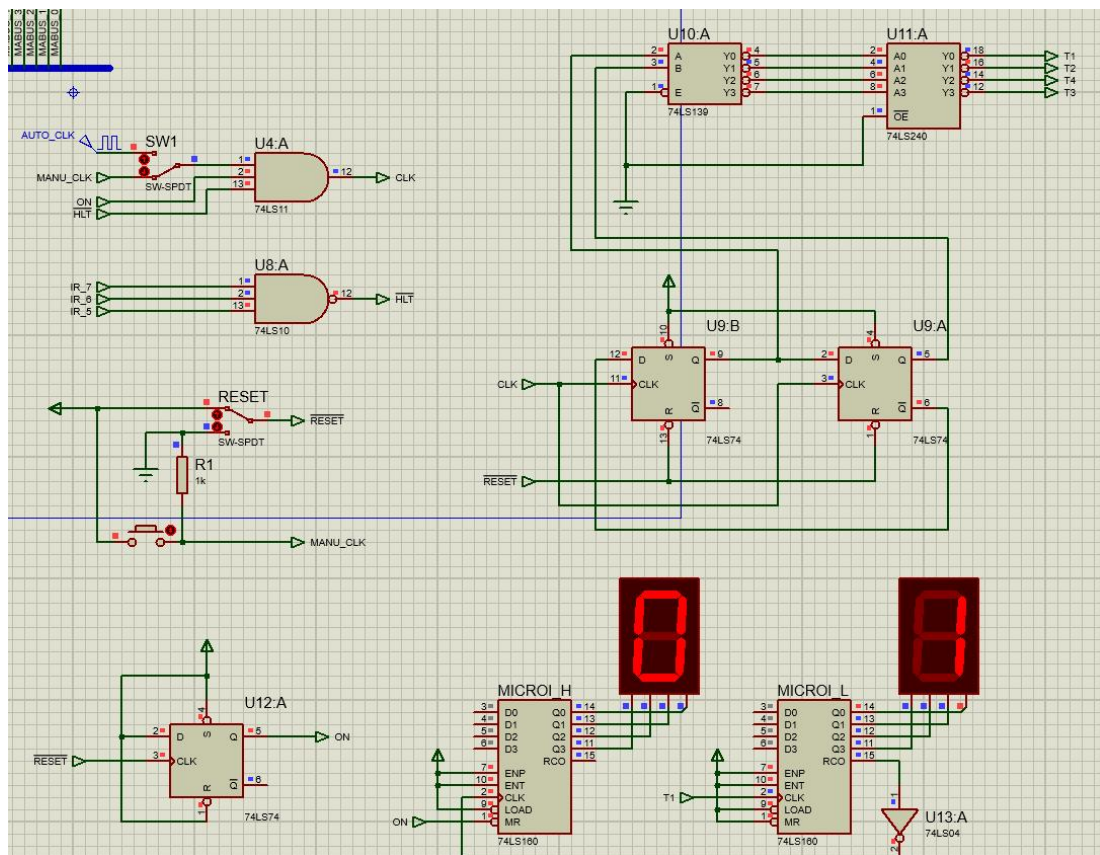
ORG      0000H
        DB      00100000B
        DB      00000110B
        DB      11101010B
        DB      00001010B
        DB      00000000B
        DB      00000000B
        DB      00000000B
        DB      00000010B
        DB      11100001B
        DB      01000000B
        DB      00000110B
END

```

令信号#RESET=1, 其上升沿跳变使初始化信号 ON=1, CLK 允许输出。初始化完成

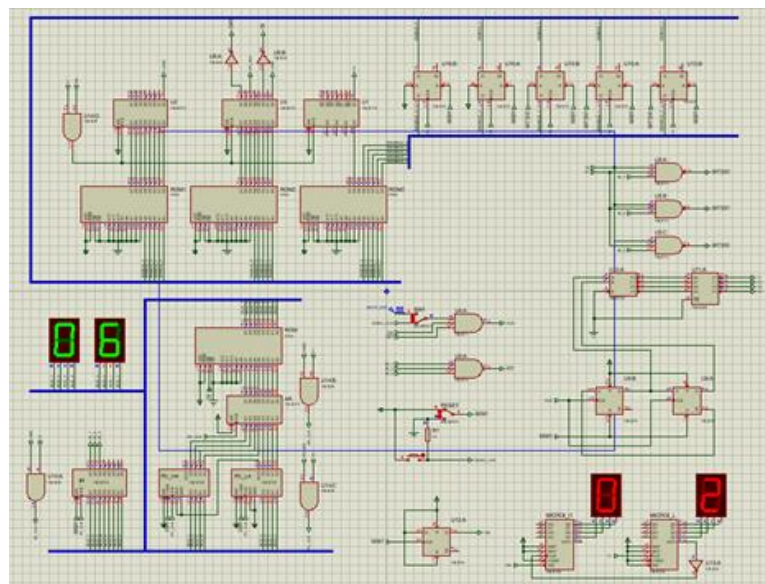
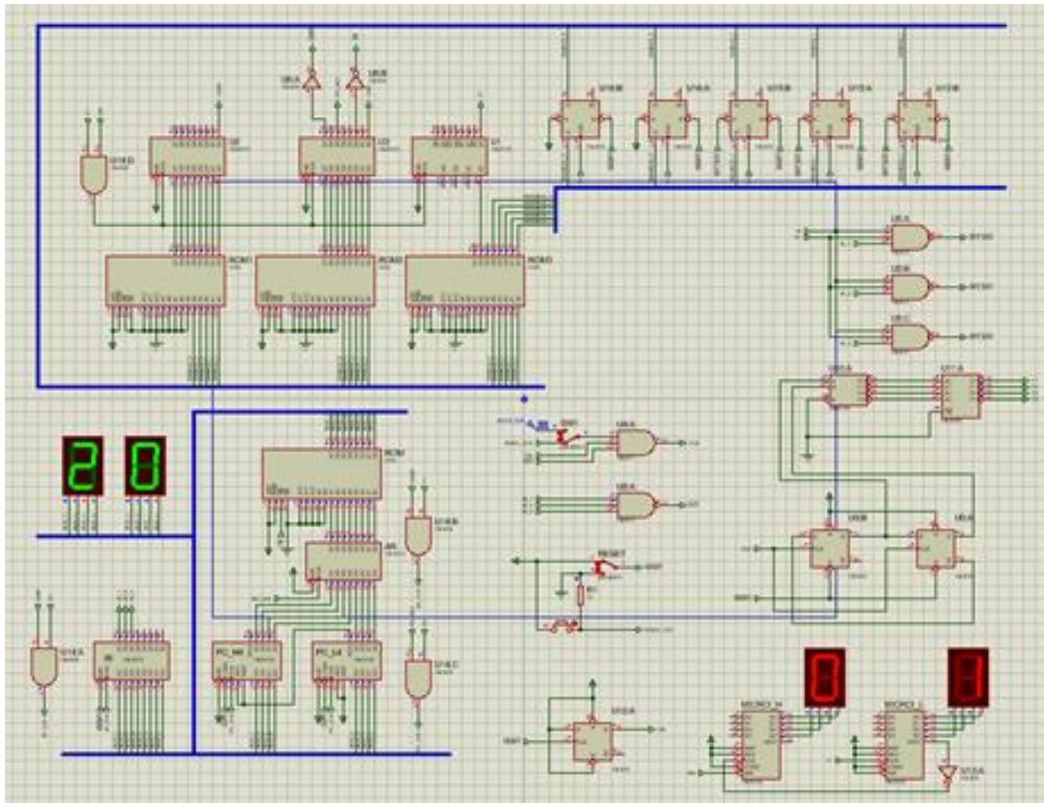
[illegible]

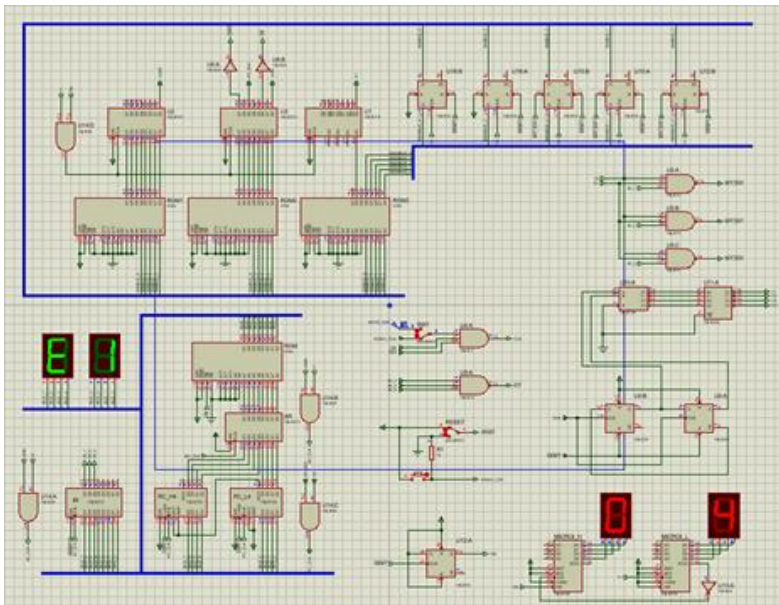
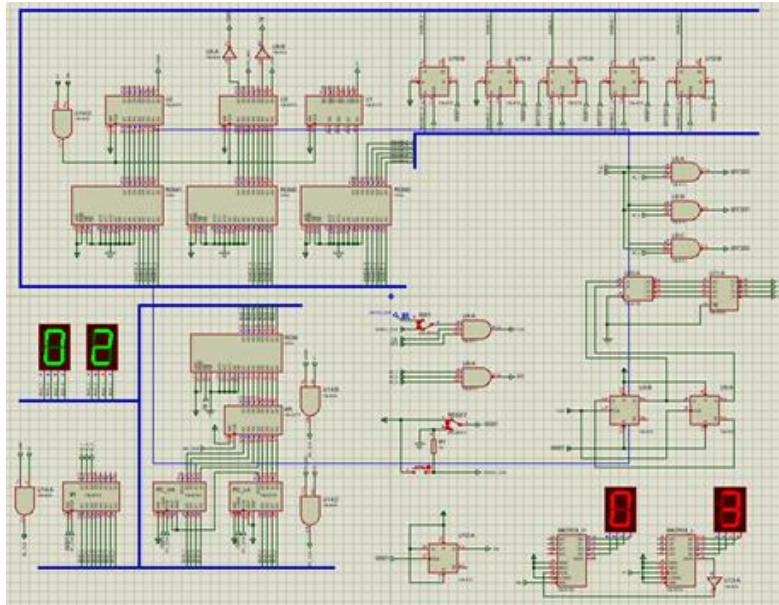




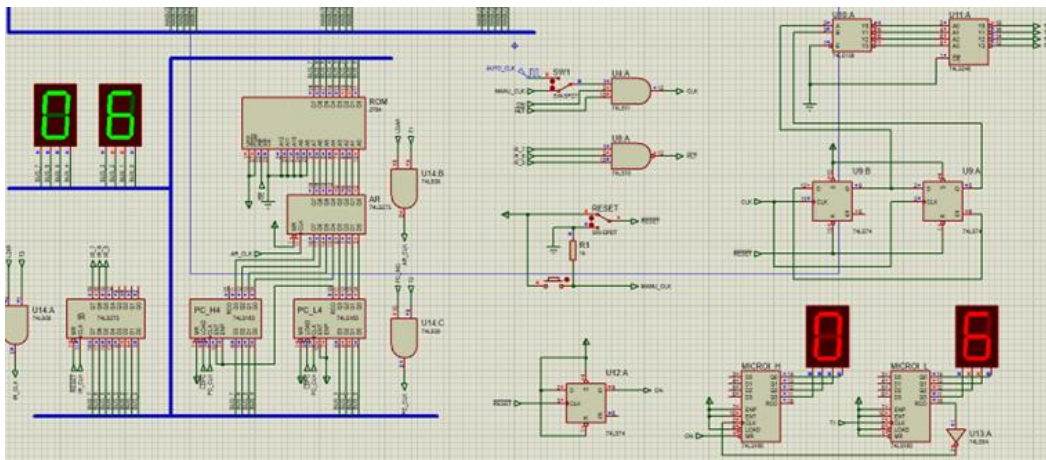
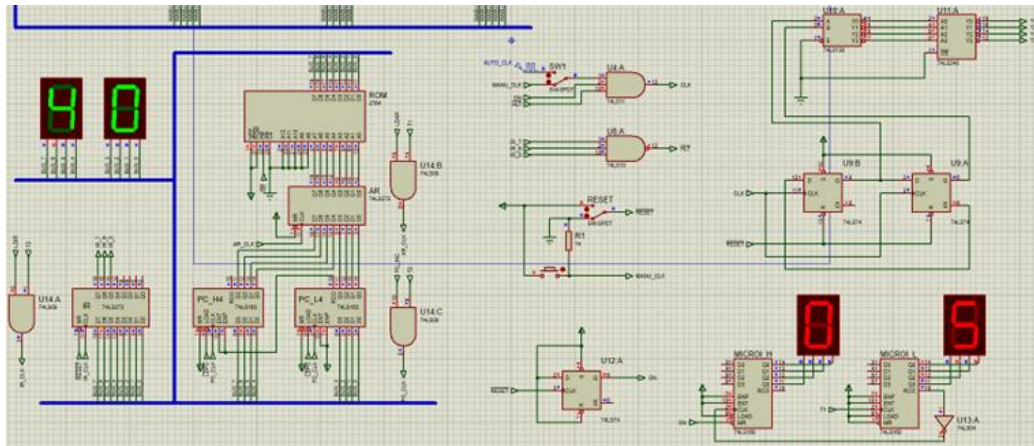
2、手动按钮 **MANUAL_CLK**，单步执行上述机器语言程序。在 **JMP1** 或 **JMP2** 指令的指令周期中，对照微程序流程图，观察每一条微指令的作用，以及单步执行的

结果例如寄存器 AR、IR、PC 及总线 BUS 上的数据)

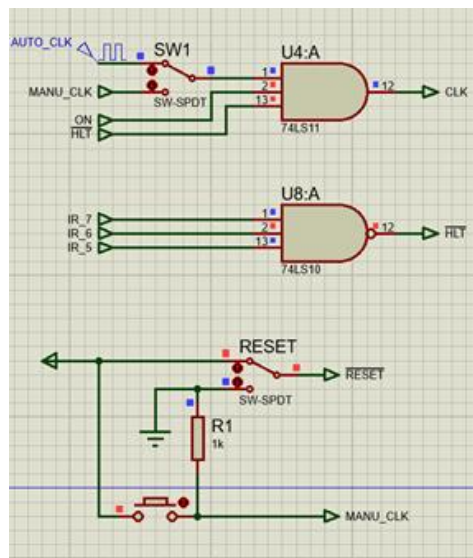


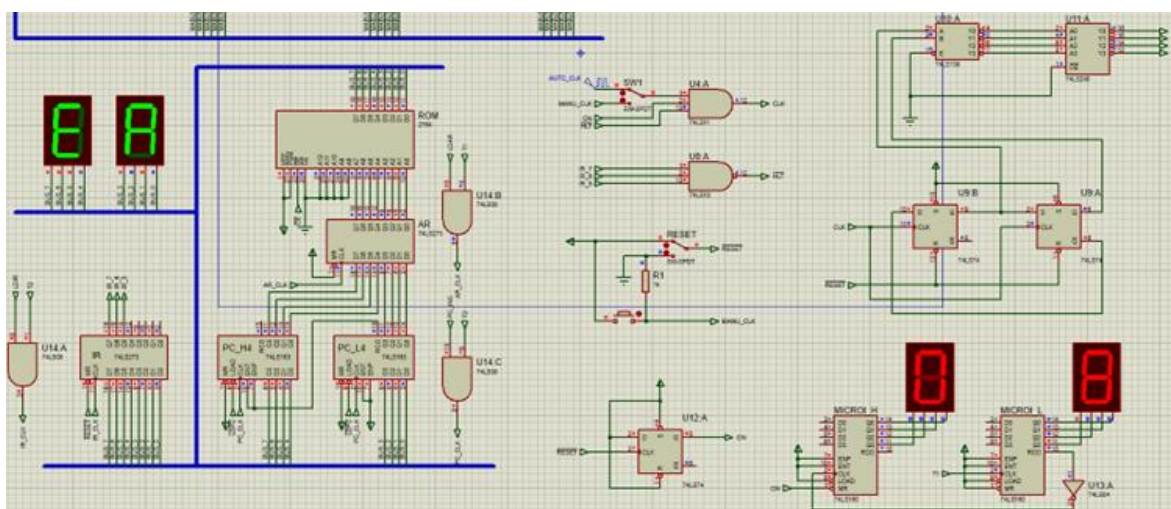
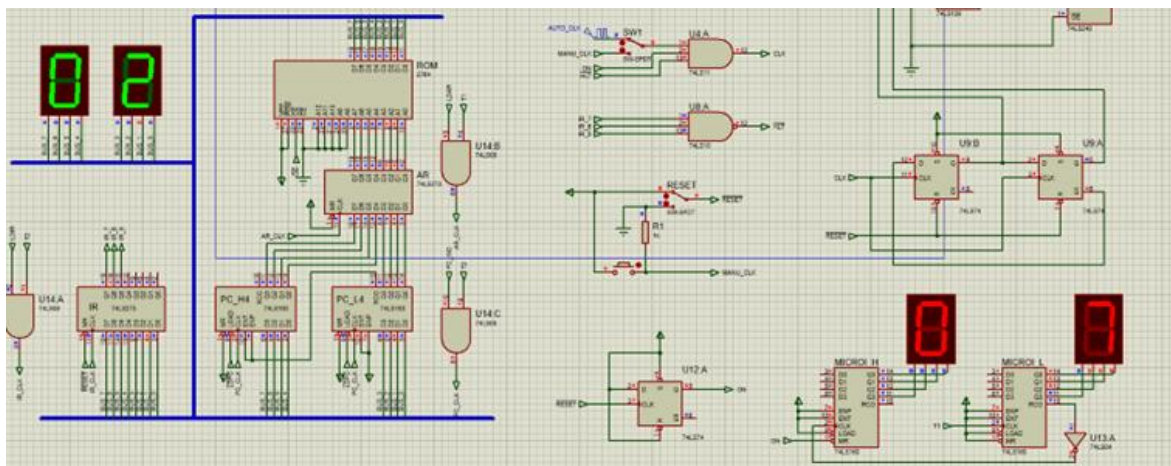


4、时钟信号 CLK 改接在 AUTO-CLK 信号源(主频 10Hz)，程序会自动运行到 HLT 指令“断点”暂停。查看“断点”处的微指令周期数指示，以及寄存器 AR、IR、PC 及总线 BUS 上的数据。使能复位信号 RESET “0→1→0”，跳出“断点”继续运行



5、时钟信号 CLK 改接在 AUTO-CLK 信号源（主频 10Hz），程序会自动运行到 HLT 指令“断点”暂停。查看“断点”处的微指令周期数指示，以及寄存器 AR、IR、PC 及总线 BUS 上的数据





五、 心得体会

通过这次实验，我深入了解了微程序控制器的组成原理和工作过程，掌握了微指令和微程序的概念，并理解了微指令与传统指令的区别与联系。此外，我还实际操作了微程序的设计过程，编写了微指令代码表，并生成了相应的 HEX 文件进行烧写，从而实现了控制存储器的操作。以下是我对本次实验的心得体会：

1. 微程序控制器的组成和工作原理：

通过实验，我清楚地了解了微程序控制器的组成部分，包括微指令寄存器、控制存储器（EPROM）、地址生成器等。微程序控制器通过微指令的执行来实现对计算机各部分的控制，这种方法具有灵活性和可扩展性。

实际操作中，通过将编写好的微程序烧写到 EPROM 中，我能够观察到控制器根据微程序的指令执行各项操作，从而直观地理解了微程序控制器的工作过程。

2. 微指令和微程序的概念：

微指令是控制器执行的最基本单位，每条微指令通常包括操作码和操作数，

用于控制计算机的具体操作。

微程序是由一系列有序的微指令组成的，用于实现复杂的指令操作。通过这次实验，我理解了微指令与传统指令之间的联系：微指令是执行传统指令的基础，每条传统指令通常对应一组微指令。

3. 微程序设计思想：

微程序设计的思想在于将复杂的指令操作分解为简单的、可重复使用的微指令，这样不仅提高了指令的执行效率，还增强了系统的灵活性。

在实验中，我尝试设计了一个“最小版本”的 CPU，重点实现了跳转功能，并通过编写汇编代码（.asm）将其转换为 HEX 文件烧写到 PROGRAM 芯片中，进一步体会到了微程序设计的实用性和灵活性。

4. 实验操作和技能提升：

本次实验要求我们编写微程序，并生成 HEX 文件进行烧写，这一过程极大地提升了我的编程技能和对计算机底层工作的理解。

通过使用 Proteus 8 Professional 进行仿真，我能够实时观察微程序控制器的工作过程，从而更加直观地理解理论知识。

5. 实验中的挑战和解决方法：

在实验过程中，我遇到了一些挑战，例如微指令代码表的编写以及 HEX 文件的生成等。通过查阅资料和与同学讨论，我逐步克服了这些困难，并成功完成了实验任务。

此外，实验中还需要准确地将微程序烧写到 EPROM 中，这要求我们具有较高的操作精度和对工具的熟练掌握。通过多次尝试和练习，我逐步掌握了这一技能。

总的来说，本次实验不仅增强了我对微程序控制器的理论理解，还提升了我的实际操作技能和问题解决能力。这次实验经验对我今后的学习产生了积极的影响。